Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. AMBIENTI DI PROGRAMMAZIONE E SVILUPPO 3
2. IL LINGUAGGIO VRML 4
3. IL LINGUAGGIO C++ 7
4. PRIMITIVE GRAFICHE 11
TESTI SUGGERITI 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Ambienti di programmazione e sviluppo

È l'insieme dei programmi che consentono la scrittura, la verifica e l'esecuzione di nuovi programmi (fasi di sviluppo). Affinché un programma scritto in un qualsiasi linguaggio di programmazione sia comprensibile (e quindi eseguibile) da un calcolatore, occorre tradurlo dal linguaggio originario al linguaggio della macchina. Questa operazione viene normalmente svolta da speciali strumenti, detti traduttori. Esistono due categorie di traduttori:

- i Compilatori traducono l'intero programma e producono il programma in linguaggio macchina;
- gli Interpreti traducono ed eseguono immediatamente ogni singola istruzione del programma sorgente.

Nel caso del compilatore, lo schema precedente viene percorso una volta sola prima dell'esecuzione.

Nel caso dell'interprete, lo schema viene invece attraversato tante volte quante sono le istruzioni che compongono il programma. I compilatori traducono automaticamente un programma dal linguaggio di alto livello a quello macchina (per un determinato elaboratore). Gli interpreti sono programmi capaci di eseguire direttamente un programma nel linguaggio scelto, istruzione per istruzione. I programmi compilati sono in generale più efficienti di quelli interpretati.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Il linguaggio VRML

A livello comunicativo si è attualmente di fronte ad un'interessante fusione di due entità apparentemente separate come Internet e RV, che ha portato alla produzione del VRML, un linguaggio per la costruzione di ambienti tridimensionali in rete. Sicuramente tale linguaggio si affermerà sempre di più in futuro, in quanto rappresenta un potente ed espressivo veicolo di informazione e comunicazione.

Il VRML (Virtual Reality Modelling Language) originariamente conosciuto con il nome di Moving Worlds, è un linguaggio descrittivo per la costruzione di ambienti tridimensionali e scene virtuali. A differenza dei più diffusi strumenti di disegno 3D e di rendering, il VRML, è nato e si caratterizza per la rappresentazione di scene tridimensionali su internet. Sulla scia del protocollo di comunicazione HTML, utilizzato per la consultazione dei siti web, il linguaggio VRML si propone pertanto come un potente veicolo di informazione e comunicazione. Tutte le pagine consultate su internet sono realizzate con il protocollo HTML (Hyper Text Mark up Language); esse sono rappresentate mediante un file ASCII (American Standard Computer Information Interchange) che contiene i comandi che permettono al browser (tipo Microsoft Internet Explorer o Netscape) di formattare testi, visualizzare immagini, richiamare animazioni ed altro ancora, tutto ciò però in ambito bidimensionale e senza che l'utente possa interagire con la scena in questione. L'obiettivo del VRML è proprio quello di portare nel web l'interazione e la terza dimensione. Esso, infatti, consente non solo di rappresentare scene artificiali contenenti oggetti tridimensionali, ma offre la possibilità di interagire all'interno di esse, modificando il punto di osservazione oppure allontanandosi o avvicinandosi da o verso gli elementi stessi. Anche il VRML è in realtà un linguaggio in modalità testo, vale a dire un semplice file in codice ASCII, ma al suo interno contiene i comandi necessari per la descrizione di una scena tridimensionale. Disponendo di un editor si possono così descrivere in sintassi opportuna gli oggetti, definendone le caratteristiche di forma, colore, le relative texture e la loro localizzazione all'interno dello spazio tridimensionale. Si potrebbe, a questo punto, obiettare che gli attuali programmi di rendering sono già in grado di

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

eseguire simili operazioni. La differenza sostanziale risiede nel fatto che i file VRML sono accessibili e condivisibili via Internet e quindi sono alla portata di milioni di utenti.

In occasione della Prima Conferenza Internazionale sul World Wide Web, tenutasi a Ginevra nel 1994, Mark Pesce e Tony Parisi presentarono una loro applicazione denominata Labyrinth, sviluppata in modo da consentire la navigazione in rete all'interno di un labirinto virtuale. In quella stessa occasione Tim Berners Lee e David Ragget, padri fondatori dell'HTML, presentarono il VRML come un linguaggio capace di rendere disponibile in rete, a breve termine ed in formato standard, la nascente rivoluzione del virtuale e della visualizzazione 3D, in modo che tutti potessero usufruirne. Tuttavia la difficoltà maggiore era legata proprio alla diversità dei calcolatori degli utenti che accedono alla rete, sia in termini di tipologie, sia di prestazioni. Un problema importante era quello di rendere disponibile uno standard per la descrizione delle scene tridimensionali, e sviluppare un browser opportuno, che interpretasse gli script realizzati in VRML. In definitiva la vera e propria risorsa necessaria per la condivisione via Internet di ambienti virtuali era e rimane legata al software di navigazione, che deve essere in grado di interpretare il codice testuale, nonché tradurlo in forma grafica ed in simulazione.

Attualmente esistono due versioni di VRML: VRML 1.0 e VRML 2.0. La grossa differenza tra le due versioni sta nel fatto che, la prima consente di descrivere mondi statici, mentre con la seconda è possibile definire la dinamica degli oggetti che compongono la scena e la modalità di interazione che questi possono avere con l'utente. Nella versione 2.0 è stata, inoltre, introdotta la possibilità di introdurre fonti audio e video in modo da rendere più realistiche le scene.

Nel dicembre del 1997 il VRML è stato riconosciuto come standard dall'Organizzazione Internazionale per la Standardizzazione ISO e dalla Commissione Internazionale per l'Elettrotecnica IEC.

Uno dei vantaggi fondamentali alla base di questo sistema di condivisione in rete consiste certamente nella semplicità dell'attrezzatura necessaria per la navigazione. Infatti sono sufficienti un mouse ed un PC di medie prestazioni per l'esplorazione del mondo virtuale 3D scaricabile dal World Wide Web. Tuttavia, bisogna tener in debito conto i limiti legati all'utilizzo di un dispositivo di

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

puntamento bidimensionale, come il mouse, in un ambiente tridimensionale. Il problema più rilevante è rappresentato dalle dimensioni dei file VRML, che, per scenari molto complessi, divengono considerevoli, in particolare se rapportate alla velocità di trasferimento dei dati, tipica della rete Internet.

La tendenza attuale, soprattutto in ambito aziendale, è quella di sviluppare, mediante software dedicati, file simili al VRML, ma di estensioni differenti. Si tratta di soluzioni ad hoc che tuttavia condizionano la trasportabilità del formato agli interessi del mercato. Esistono già in fase di sperimentazione dei sistemi che consentono di creare ambienti virtuali interattivi in rete, e che cercano di risolvere il problema della scomponibilità, ovvero dell'abilità di trasportare dinamicamente oggetti e comportamenti in un assegnato mondo virtuale, anche se tali oggetti sono stati creati originariamente come parte di una scena completamente differente. Si cita a tal proposito Ygdrasil, software che lavora sull'interazione automatica tra mondi, senza che siano necessarie modifiche di codice nel trasportare parti di scene virtuali in nuovi ambienti. Nell'ottica delle applicazioni di RV in ambito industriale, un sistema completamente componibile accelererebbe la creazione di scene complesse o mondi virtuali condivisi, distribuendo in maniera parallela il lavoro. Un approccio simile ben si configura in un orientamento progettuale improntato sul concurrent engineering.

Il trend attuale resta tuttavia quello di definire un linguaggio che consenta ai diversi utenti in rete di interagire contemporaneamente all'interno di un ambiente virtuale, mediante avatar, ossia mediante la loro stessa rappresentazione fisica. La realizzazione di tali modalità di utilizzo è già resa possibile mediante applicazioni esterne che impieghino il VRML, sia attraverso la sua stessa interfaccia, oppure mediante quella Java.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Il linguaggio C++

Nato nel 1983 dall'ingegno di Bjarne Stroustrup, allora ricercatore presso AT&T, C++ è tra i primi 5 linguaggi più utilizzati al mondo. Il nome è un gioco di parole con un costrutto del C (dove il doppio segno più è l'operatore di autoincremento, che incrementa di una unità il valore di una variabile) insieme con la comune convenzione di aggiungere un segno più per indicare una versione potenziata.

I campi di applicazione sono i più svariati: dal gaming alle applicazioni real-time, dai componenti per sistemi operativi ai software di grafica e musica, dalle app per cellulari ai sistemi per supercomputer. Praticamente, C++ è ovunque. Negli anni si sono avvicendate diverse versioni di questo linguaggio di programmazione, introducendo via via diverse modifiche ed alcune caratteristiche che ne fanno ancora un linguaggio assolutamente importante e moderno.

L'ambiente di sviluppo di un applicativo virtuale ormai raggiunge livelli di complessità che sfuggono alle possibilità di un singolo programmatore o anche di un piccolo gruppo. La prospettiva emergente, che nasce da esigenze di riduzione dei tempi di sviluppo e di aumento della complessità del progetto, è quella di definire parti di codice del software da progettare mediante team diversi, con specializzazioni differenti. Il linguaggio di programmazione più evoluto al momento per questo tipo di applicazioni è il C++ con la sua logica di programmazione orientata ad oggetti, (Object Oriented Programming — OOP). Generalmente quando si programma in OOP si suddivide un problema in sottogruppi di parti conciate, che contengono sia il codice, sia i dati. I sottogruppi, organizzati in modo tale da essere indipendenti, vengono chiamati oggetti. Ogni oggetto è legato gerarchicamente ad altri oggetti solo mediante la sua interfaccia pubblica. Infatti, tutto ciò che è all'interno dell'oggetto non interessa all'utilizzatore dell'oggetto stesso. In questo modo programmatori differenti possono utilizzare oggetti sviluppati da altri, ignorando il codice sorgente contenuto al loro interno.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il C++ si evolve continuamente, sebbene con una certa inerzia, ed include un numero sempre maggiore di costrutti che ne hanno esteso l'applicabilità ad altri paradigmi, oltre che a quello della programmazione orientata agli oggetti, ad esempio la programmazione funzionale.

Nello standard del 2011 infatti sono state introdotte nuove caratteristiche ed il set di istruzioni è stato notevolmente ampliato. L'ultima versione è stata pubblicata nel 2014, con l'introduzione di piccole migliorie rispetto al precedente. Attualmente una nuova versione è in fase di discussione, e la data finale di rilascio è prevista per il 2017.

IL C++ è quindi un linguaggio in continua evoluzione, la cui compatibilità con il linguaggio C deve essere relegata al solo supporto garantito per la compilazione di applicazioni pre-esistenti o componenti legacy.

L'uso corretto dei costrutti del linguaggio può rendere agevole lo sviluppo di applicazioni complesse e articolate, nonchè portabili su più piattaforme, massimizzando allo stesso tempo l'efficienza durante l'esecuzione e la resilienza del codice all'aggiunta di nuove funzionalità.

La programmazione a oggetti (o programmazione "orientata agli oggetti") è ormai affermata e trova sempre maggiore successo nell'ambito della progettazione e dello sviluppo del software. Si tratta di un modo di pensare in modo astratto ad un problema, utilizzando concetti del mondo reale piuttosto che concetti legati al computer. In tal modo si riescono a comprendere meglio i requisiti di progetto, si ottengono delle soluzioni più chiare e ne risultano sistemi software in cui la manutenzione è decisamente più facile. Gli oggetti costituiscono il costrutto fondamentale di tale filosofia. Un oggetto è una singola entità che combina sia strutture dati che comportamenti: i dati sono visti come proprietà dell'oggetto e le procedure per accedere ai dati ed effettuare elaborazioni sono viste come metodi (le "vecchie" funzioni). Gli oggetti, in sostanza, possono essere visti come dei mattoni che possono essere assemblati e che hanno un alto potenziale di riutilizzo. Infatti uno dei benefici principali ed uno dei motivi per cui è stata sviluppata la programmazione ad oggetti è proprio quello di favorire il riutilizzo del software sviluppato. La programmazione a oggetti non è una prerogativa del C++, anzi è trasversale alla quasi totalità dei linguaggi moderni,

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

e risulta sempre più preponderante rispetto alle altre tecniche di modellazione e di realizzazione del software.

L'evoluzione dello standard indica una chiara tendenza all'ampliamento del set di strumenti a disposizione del programmatore, che possano semplificare la scrittura ed il mantenimento del codice. Al fine di acquisire una buona conoscenza del linguaggio C++ è quindi bene tenere a mente che non esistono costrutti equivalenti, nuovi o vecchi, utili o superflui. Tutti insieme essi concorrono nella definizione del linguaggio e delle sue potenzialità e la loro corretta applicazione è ciò che distingue un codice corretto sotto il mero aspetto sintattico e funzionale, da uno robusto ed espandibile.

Da un punto di vista storico, è bene sottolineare che la diffusione di questo paradigma al di fuori degli ambiti di ricerca accademici è stata in qualche modo agevolata dall'apparizione di C++ nel panorama dei linguaggi di programmazione. Non si può quindi non riconoscere un forte legame tra le due cose, al punto che la conoscenza di C++ può essere importante per la comprensione dei principi generali della programmazione a oggetti.

Tuttavia a differenza di altri linguaggi come Java o C#, C++ ha un forte legame con tecniche di programmazione antecedenti, come la programmazione procedurale, vista la sua diretta parentela con il linguaggio C. Cosa comporta questo nell'ottica di voler avvalersi dei principi generali della programmazione a oggetti in C++? Semplicemente il fatto che in C++ è più facile violare le regole del paradigma, pur avendo a disposizione tutti gli strumenti per applicarle.

Sebbene la programmazione a oggetti permetta di raggiungere un elevato livello di concettualizzazione, non è infatti semplice applicarne i principi in una maniera che consenta effettivamente di preservare la validità di tutte le sue promesse, soprattutto la riusabilità e l'espandibilità del software, che sono le più importanti. Sotto quest'ottica, C++ è probabilmente uno dei linguaggi più usati e allo stesso tempo fraintesi, tuttavia è anche il più vicino ai livelli prestazionali dei linguaggi appartenenti alla generazione precedente, pur integrando i benefici della moderna programmazione a oggetti.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Esempio del programma hello world che utilizza la libreria standard C++, il flusso di output cout per mostrare un messaggio e termina, restituendo ("return") un valore alla funzione chiamante, in questo caso il sistema operativo che ha lanciato il programma.

```
#include <cstdlib>  // per EXIT_SUCCESS
#include <iostream> // per std::cout e std::endl
int main()
{
   std::cout << "Hello World!" << std::endl;
  return EXIT_SUCCESS;
}
```

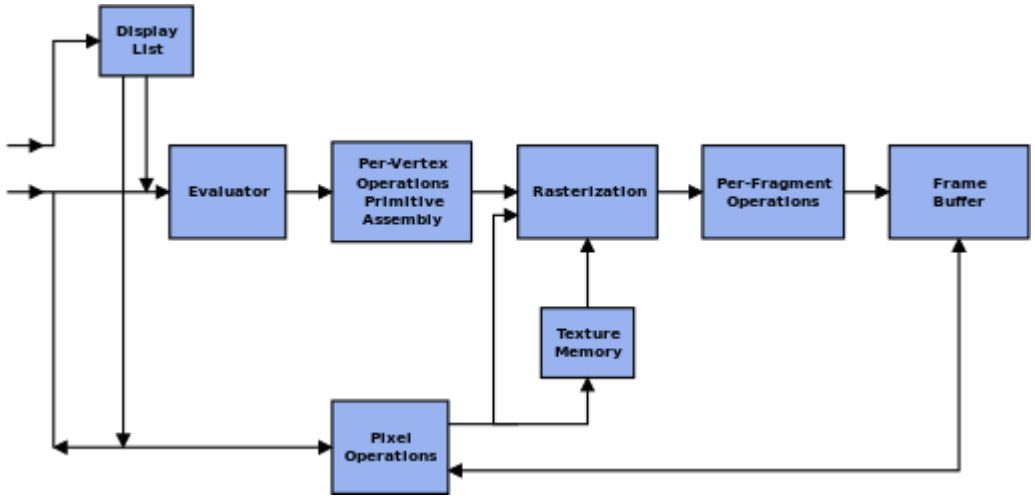
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Primitive grafiche

Per la stesura di applicativi destinati alla grafica computerizzata risulta necessario utilizzare un toolkit 3D che, al suo interno, abbia implementate primitive grafiche; ciò per non dover costruire ogni volta la singola immagine da visualizzare, che inoltre non è, in questo modo, facilmente modificabile. Bisogna dunque soddisfare le necessità di standard grafici. Al momento, se si escludono le librerie grafiche Microsoft DirectX, lo standard per la grafica professionale è rappresentato da OpenGL, che dalla sua introduzione nel 1992 si è rapidamente affermato su tutte le piattaforme.

OpenGL assolve a due compiti fondamentali:

- nascondere la complessità di interfacciamento con acceleratori 3D differenti, offrendo al programmatore una API unica ed uniforme;
- nascondere le capacità offerte dai diversi acceleratori 3D, richiedendo che tutte le implementazioni supportino completamente l'insieme di funzioni OpenGL, ricorrendo ad un'emulazione software se necessario.



Il compito di OpenGL è quello di ricevere primitive come punti, linee e poligoni, e di convertirli in pixel (rasterizing o rasterizzazione). Ciò è realizzato attraverso una pipeline grafica nota

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

come OpenGL state machine. La maggior parte dei comandi OpenGL forniscono primitive alla pipeline grafica o istruiscono la pipeline su come elaborarle. Prima dell'introduzione di OpenGL 2.0, ogni stadio della pipeline realizzava una funzione fissa ed era configurabile solo entro certi limiti, ma dalla versione 2.0 molti stadi sono totalmente programmabili attraverso il linguaggio GLSL.

Storicamente, OpenGL ha esercitato una notevole influenza sullo sviluppo degli acceleratori 3D, promuovendo un livello base di funzionalità che è oggi comune nelle schede video destinate al grande pubblico:

- punti, linee e poligoni disegnati come primitive base;
- una pipeline per il transform and lighting;
- Z-buffering;
- Texture mapping;
- Alpha blending.

Una delle caratteristiche più apprezzate in ambito professionale è la retrocompatibilità tra le diverse versioni di OpenGL: programmi scritti per la versione 1.0 della libreria devono funzionare senza modifiche su implementazioni che seguono la versione 2.1.

OpenGL è una API procedurale che opera a basso livello, che richiede al programmatore i passi precisi per disegnare una scena. Questo approccio si pone in contrasto con le API descrittive ad alto livello le quali, operando su struttura dati ad albero (scene graph), richiedono al programmatore solo una descrizione generica della scena, occupandosi dei dettagli più complessi del rendering. La natura di OpenGL obbliga quindi i programmatori ad avere una buona conoscenza della pipeline grafica stessa, ma al contempo lascia una certa libertà per implementare complessi algoritmi di rendering.

A differenza di OpenGL, in cui i modelli dei dati vengono utilizzati esclusivamente come vettori, l'avvento del sistema open source OpenSG, ovvero Open Scene Graph, scritto in C++, ha consentito di poter organizzare gerarchicamente, in una struttura ad albero, i dati relativi agli oggetti rappresentati, disponendoli in base alla loro dipendenza. In questo modo, nel caso in cui uno stesso dettaglio ricorra ripetutamente, esso va modellato una sola volta per poi essere

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

trasformato nella scena visualizzata e disposto in corrispondenza di nodi opportuni dello scene graph. Ciò consente di poter visualizzare molto più velocemente gli oggetti, calcolando in effetti esattamente solo la porzione della scena totale che compare nel campo di vista dell'utente, nascondendo quelle che invece non vi rientrano. Il sistema, inoltre, consente di variare il livello di dettaglio, o LODI, in base alla distanza osservatore-oggetto, così da ridurre i tempi di rendering. OpenSG è un sistema per creare programmi di grafica in tempo reale, ad esempio per applicazioni di realtà virtuale.

Sulla base delle modifiche introdotte con OpenSG, positive sia in termini di flessibilità, sia di trasferibilità e di compatibilità tra sistemi operativi, è stato sviluppato OpenInventor, piattaforma software object oriented per la creazione di applicazioni interattive grafiche in 3D che sfruttano le caratteristiche degli attuali acceleratori, con un minimo sforzo di programmazione. OpenInventor fornisce una libreria di elementi base, detti nodi, che possono essere utilizzati, modificati ed estesi per soddisfare nuove richieste. Chi utilizza OpenInventor deve focalizzare la sua attenzione esclusivamente sulla costruzione di oggetti 3D, dimenticando le interfacce di manipolazione e visualizzazione, al contrario di quanto accade per OpenGL. I nodi di OpenInventor comprendono primitive geometriche, quali forma, proprietà e raggruppamento degli oggetti. Tutte le informazioni vengono raccolte in un database chiamato scene graph o albero gerarchico della scena, come già accade in OpenSG. Proprio grazie a tale struttura e ad una simile organizzazione dei dati è possibile eseguire sugli oggetti anche operazioni, diverse dal rendering, quali la selezione, la manipolazione e la modifica. In definitiva non solo è possibile variare il punto di osservazione della scena, che non altera le coordinate dei punti degli oggetti, ma si può anche modificare la posizione degli oggetti rispetto alla terna di riferimento prefissata, ottenendo in tal modo una variazione delle loro coordinate. Tutto ciò avviene tramite altri elementi appositamente costruiti, chiamati manipolatori. Si tratta di particolari tipi di nodi in cui sono definite le specifiche per la manipolazione diretta e interattiva degli oggetti 3D. Essi possono essere correlati a particolari eventi come l'attivazione di un'interfaccia semplice e naturale per la rotazione o traslazione degli oggetti. Le interfacce consentono di interagire con la scena e di esaminarne le caratteristiche. Inoltre si

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

possono introdurre alcuni elementi di fondamentale importanza per la costruzione di un ambiente virtuale, come un editor per i materiali, le luci direzionali o le camera, ossia punti di osservazione prefissati all'interno della scena. Lo Scene Viewer permette di analizzare la scena secondo diverse modalità, ognuna delle quali offre strumenti di navigazione di diverso tipo. Oltre alla modalità di default caratteristica di tutti i sistemi CAD, prevede infatti quella fly, walk o piane, che consentono, rispettivamente, di visualizzare la scena dall'alto, camminare all'interno di essa o sezionare oggetti per poterne osservare le caratteristiche interne.

OpenGL è un basso livello di interfaccia di programmazione di applicazione che prende gli elenchi dei semplici poligoni e li rende il più rapidamente possibile. Per fare qualcosa di più pratico come "disegnare una casa", il programmatore deve abbattere l'oggetto in una serie di semplici istruzioni OGL e inviarli nel motore per il rendering. Un problema è che la prestazione OGL è molto sensibile al modo in cui queste istruzioni vengono inviate nel sistema, richiedendo all'utente di sapere le istruzioni per l'invio, in quale ordine e costringendolo ad abbattere attentamente i dati per evitare l'invio di oggetti che non siano anche visibili nell'immagine risultante.

Open Inventor (OI) è stato scritto per risolvere questo problema, e di fornire uno strato di base comune per iniziare a lavorare con oggetti che potrebbero essere sottoclassi di un certo numero di forme come cubi e poligoni, e quindi facilmente modificati in nuove forme. Il "mondo" da disegnare è posto in una scena grafica gestita da OI, con il sistema di applicazione si ottiene l'abbattimento degli oggetti nel grafico automaticamente. OI comprende anche una serie di oggetti e sistemi di controllo per la loro applicazione alla scena, rendendo le attività di interazione comuni più facili. Infine, OI ha anche fornito un formato di file comune per la memorizzazione di "mondi", e il codice per salvare o caricare un mondo da questi file automaticamente. Applicazioni 3D di base potrebbero quindi essere scritte in poche centinaia di righe sotto OI, legando insieme le parti del kit con il codice "colla".

Lo svantaggio di OI è che tende ad essere più lento del codice scritto a mano, come le attività in 3D sono notoriamente difficili da far eseguire bene senza il trascinamento dei dati della scena grafica fatta a mano. Un altro problema pratico da segnalare è che poteva essere utilizzato

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

solo con il proprio formato di file, costringendo gli sviluppatori a scrivere convertitori da e verso il sistema interno. I suoi obiettivi principali sono migliore programmazione, convenienza ed efficienza.

OpenInventor fornisce una serie di nodi che è possibile assemblare in un scene graph, nonché la possibilità di crearne di nuovi. Questi vantaggi hanno reso OpenInventor una delle librerie grafiche più diffuse nella comunità scientifica che opera nella RV.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Testi suggeriti

- Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley
- Bjarne Stroustrup, C++. Linguaggio, libreria standard, principi di programmazione
- OpenGL Architecture Review Board; Dave Shreiner. OpenGL Reference Manual: The Official Reference Document to OpenGL, Version 1.4 (4th Edition), Addison-Wesley Professional, 2004.
- Pellegrinetti G., "Introduzione alla realtà virtuale".
- Pape D., Anstey J., Dolinsky M.,Dambik E.J., "Ygdrasil-a framework for composing shared virtual words", in: Future Generation Computer System, Volume 19, Number 6, August 2003.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).